Curse of Dimensionality & PCA

The process, the maths, an example and Colab code

Concept | Process | Example | Code

Data Science Journey • Part 4 →

Curse of Dimensionality

When more features make things WORSE

As the number of features (dimensions) grows, data becomes sparse, distances lose meaning, and models struggle to generalise.

More features ≠ better model

Example

Predicting house price: 5 features is fine.
5,000 features with 500 houses = trouble.

Term

Coined by Richard Bellman (1957)

Why It Hurts

Four big problems

1. Data becomes sparse

10 points per axis needs 10^d samples:
100 for 2 features, 10 BILLION for 10

2. Distances lose meaning

nearest and farthest points look equally far
(KNN, K-means, clustering all suffer)

3. Overfitting

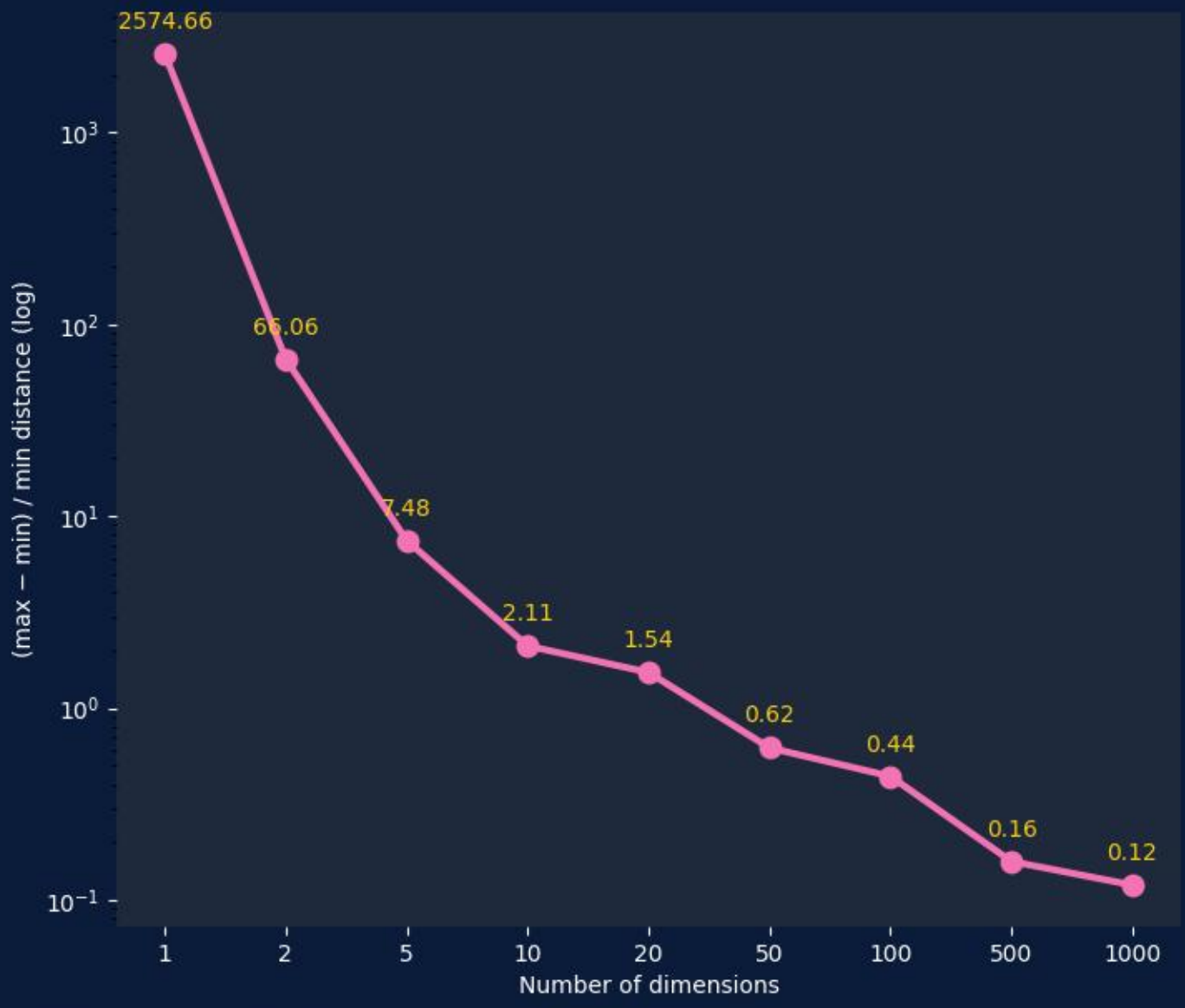
features ≥ samples → model memorises noise

4. Slow and costly

more memory, longer training, harder to visualise

Proof: Distances Collapse

1,000 random points, relative gap between farthest and nearest



At 1 dimension the farthest point is 2,575× further than the nearest; at 1,000 dimensions only 0.12×. Everything looks equally far.

How to Beat the Curse

Reduce the dimensions

Feature selection

keep only the useful original columns

Feature extraction (PCA)

combine columns into fewer NEW ones

Regularisation

Lasso / Ridge shrink useless weights

Get more data

helps, but needed amount grows exponentially

Domain knowledge

drop or merge redundant features by hand

What is PCA?

Principal Component Analysis

An unsupervised, linear technique that rotates the data onto NEW axes (principal components) ordered by how much variance they capture.

PC1

direction of maximum variance

PC2

next best direction, perpendicular to PC1

Keep top k

drop the rest → fewer features, little loss

Components are uncorrelated linear mixes of the original features

The PCA Process

5 steps from raw data to components

1. Standardise

$z = (x - \text{mean}) / \text{std}$ (equal scales)

▼

2. Covariance matrix

$C = X^{T}X / (n-1)$ (how features vary together)

▼

3. Eigen-decomposition

$C\,v = \lambda\,v$ (directions v, importance λ)

▼

4. Sort & choose k

order by λ, keep top k (e.g. 95% variance)

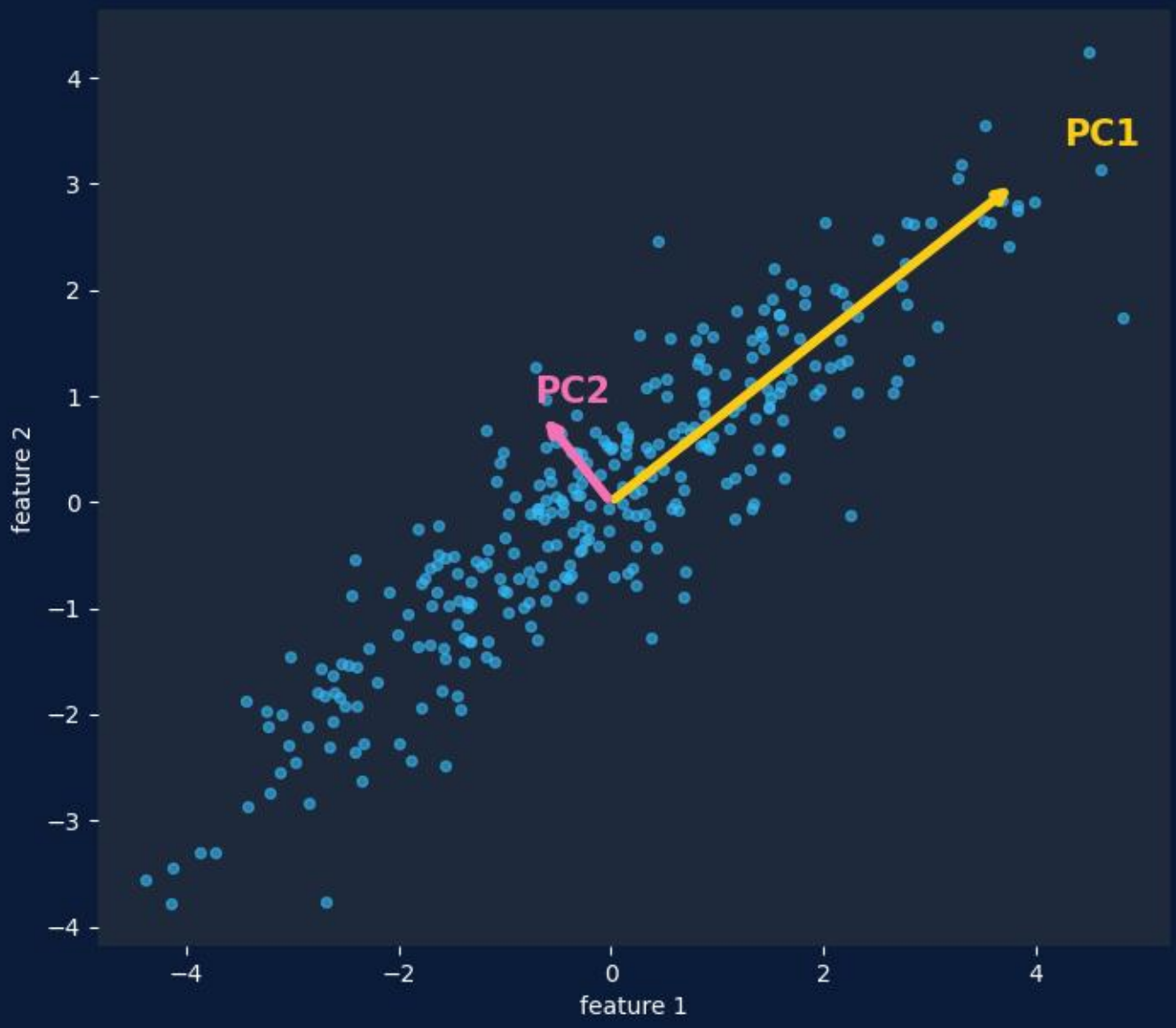
▼

5. Project the data

$Z = X \cdot W$ (new, smaller dataset)

PCA Intuition in 2D

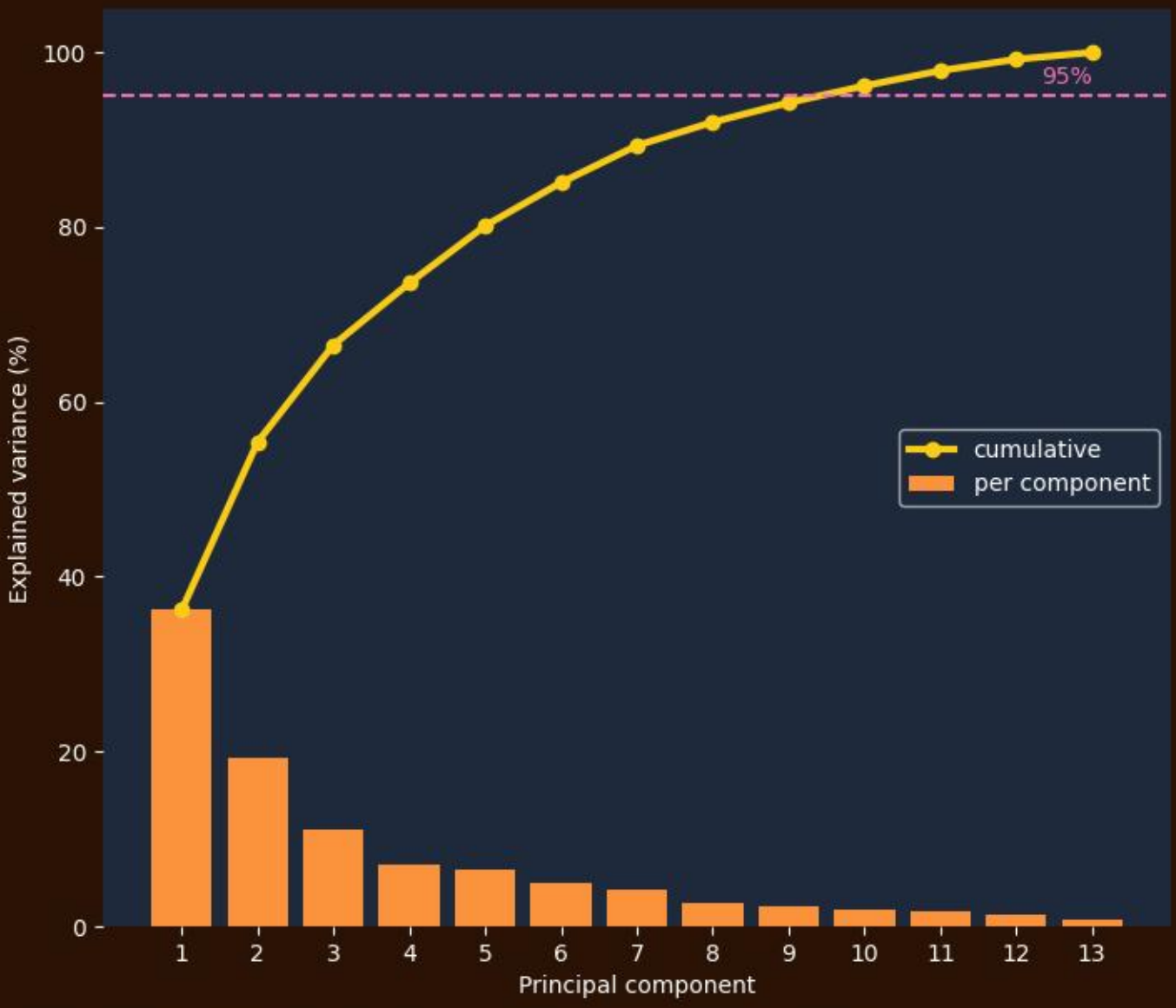
Rotate the axes to follow the data's spread



PC1 captures 95% of the variance, PC2 only 5%.
Drop PC2 and 2 features become 1 with little loss.

Example: Wine Dataset

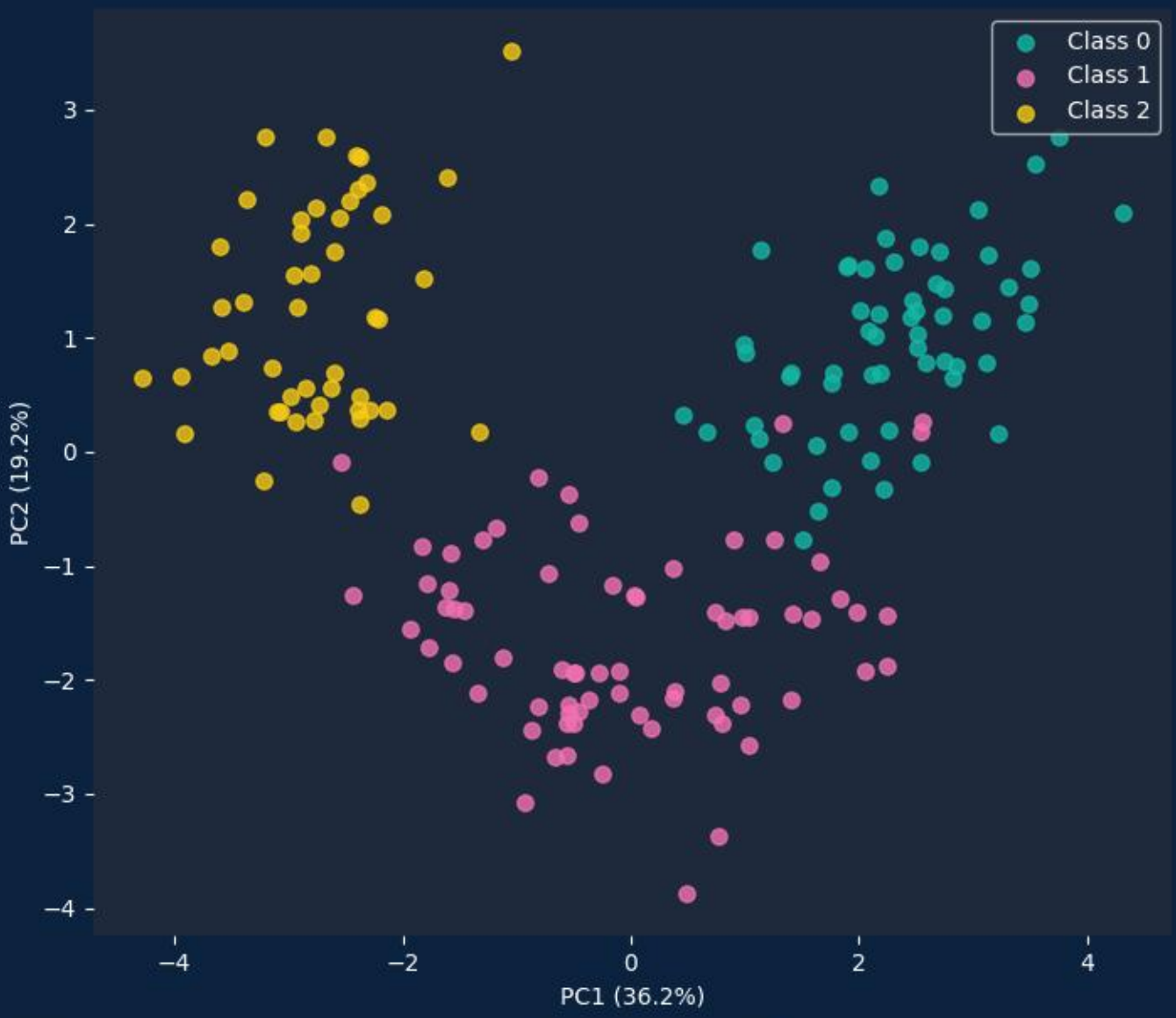
178 wines, 13 chemical features → how many components?



PC1 + PC2 = 55.4% of variance. 10 components keep 95% (13 → 10 features).

13 Features → 2D Map

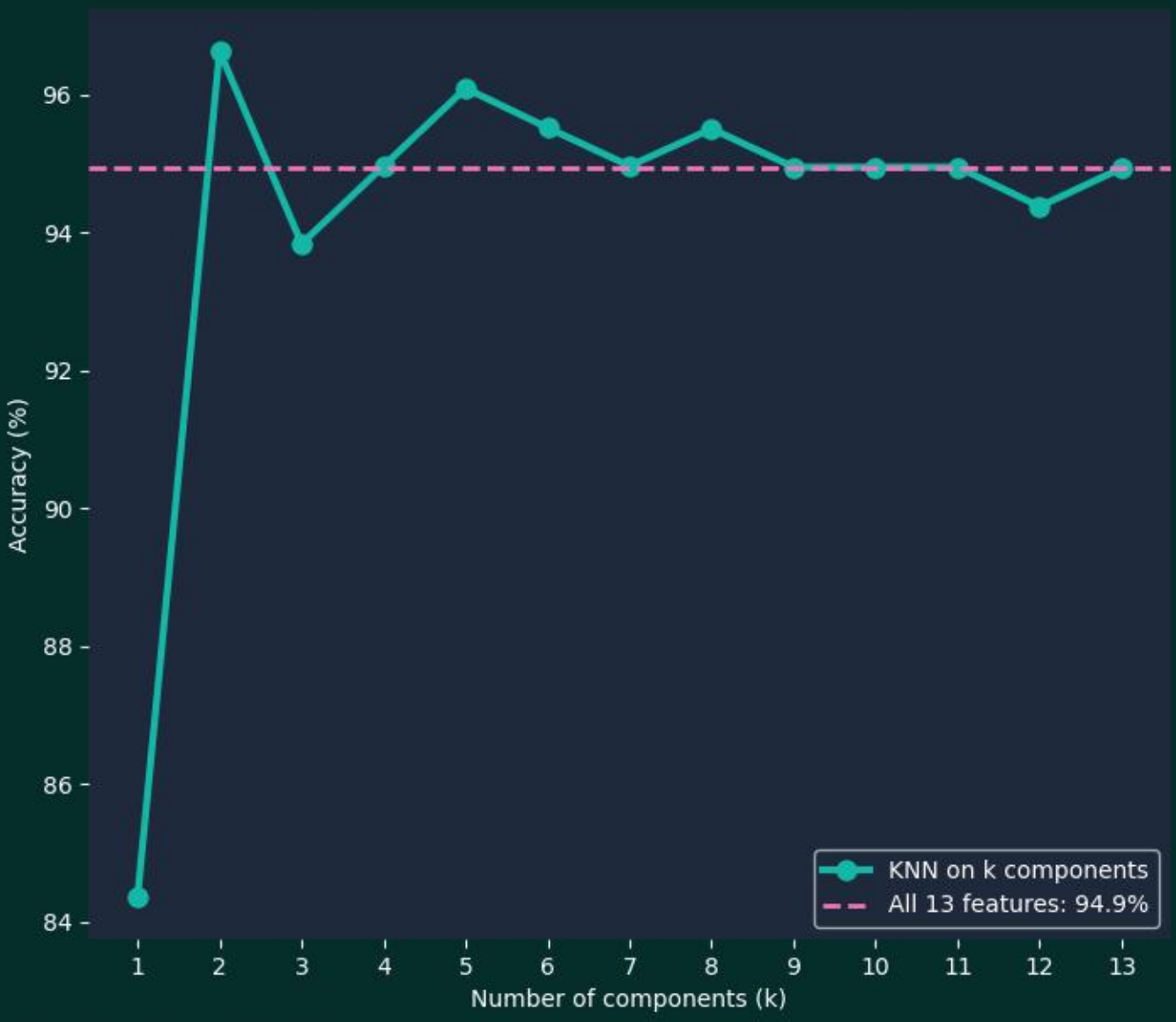
Wine classes separate clearly on just PC1 and PC2



PCA never saw the labels, yet the 3 wine types still form clear groups.

Does PCA Hurt Accuracy?

KNN, 5-fold CV, PCA fitted inside each fold



Best: k = 2 with 96.6% accuracy vs 94.9% using all features.
Fewer dimensions, similar or better performance.

PCA in Python: sklearn

Google Colab • Wine dataset

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X, y = load_wine(return_X_y=True, as_frame=True)
print(X.shape)                    # (178, 13)

# 1. Standardise (PCA needs equal scales)
Xs = StandardScaler().fit_transform(X)

# 2. Fit PCA with all components
pca = PCA().fit(Xs)
ev  = pca.explained_variance_ratio_
cum = np.cumsum(ev)
print(ev.round(3)); print(cum.round(3))

# 3. Choose k: keep 95% of the variance
pca95 = PCA(n_components=0.95)
Xp = pca95.fit_transform(Xs)
print(Xp.shape)

# 4. Visualise the first 2 components
Z = PCA(2).fit_transform(Xs)
plt.scatter(Z[:,0], Z[:,1], c=y, cmap="viridis")
plt.xlabel("PC1"); plt.ylabel("PC2"); plt.show()

# 5. Loadings: which features drive each PC?
load = pd.DataFrame(pca.components_[:2].T,
        index=X.columns, columns=["PC1","PC2"])
print(load.round(2))
```

PCA From Scratch + Check

Google Colab • NumPy and sklearn

```
# PCA from scratch with NumPy
Xc  = Xs - Xs.mean(axis=0)          # centre the data
cov = np.cov(Xc, rowvar=False)      # covariance matrix
vals, vecs = np.linalg.eigh(cov)    # eigen-decomposition
idx = np.argsort(vals)[::-1]        # sort by importance
vals, vecs = vals[idx], vecs[:, idx]

k = 2
W = vecs[:, :k]                     # top-k eigenvectors
Z = Xc @ W                          # project the data
print((vals / vals.sum())[:k].round(3))

# Does PCA hurt accuracy? (PCA inside the pipeline
# avoids data leakage during cross-validation)
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline

for k in [2, 5, 8, 13]:
    pipe = make_pipeline(StandardScaler(), PCA(k),
                         KNeighborsClassifier())
    print(k, cross_val_score(pipe, X, y, cv=5).mean().round(3))

# Tips
# - ALWAYS scale before PCA
# - fit PCA on train data only, then transform test
# - plot explained variance to choose k
```

Pros, Cons & When to Use

Pros

reduces noise & overfitting, speeds up training,
removes multicollinearity, enables 2D/3D plots

Cons

components are hard to interpret, only LINEAR,
sensitive to scale and outliers

Use when

many correlated numeric features, visualisation,
before KNN / SVM / clustering

Avoid when

you need interpretable features, or the structure
is non-linear (try t-SNE / UMAP / autoencoders)

Cheat Sheet

Curse of dimensionality

more features → sparse data, useless distances

PCA in one line

rotate to uncorrelated axes, keep the top-variance ones

Golden rules

scale first • fit on train only • pick k by explained variance

Selection vs PCA

selection keeps old features; PCA builds new ones

Found this useful?

Like

Repost

Follow for the next topic: Model Building

#DataScience #PCA #CurseOfDimensionality #MachineLearning #Python